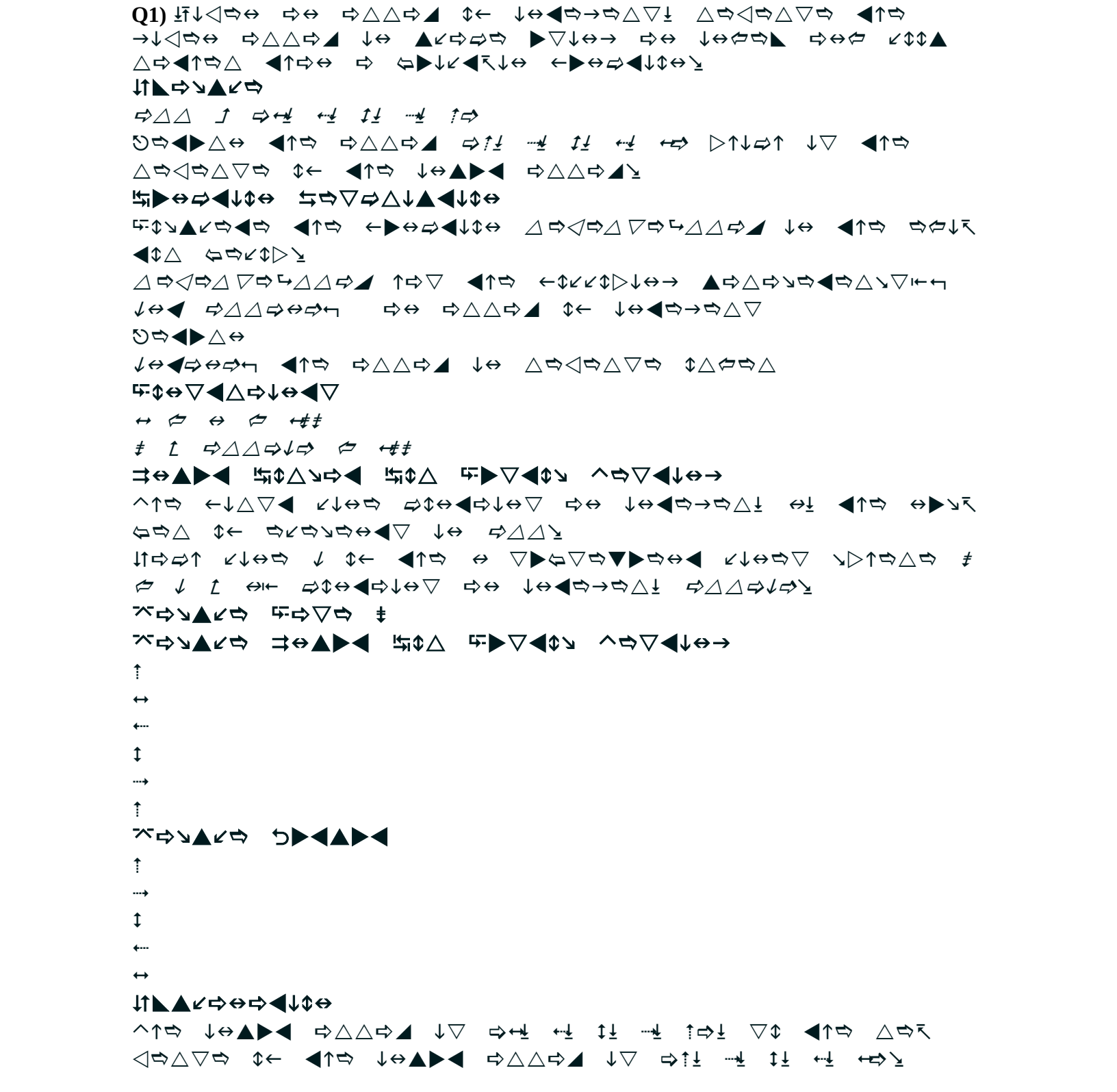
Week 15

Q1) 

🡅🡆🡖🡅🡗🡆 🡇🡆🡇🡆 🡘
🡅🡆🡖🡅🡗🡆 🡒🡘🡅🡅🡄 🡇🡆🡇🡄🡖 🡅🡆🡇🡄🡓🡘🡒
🡒
🡘🡗
🡘🡻
🡙🡘
🡒🡱
🡅🡆🡖🡅🡗🡆 🡔🡆🡄🡅🡆🡄
🡒🡱
🡙🡘
🡘🡻
🡘🡗
🡻🡱🡇🡅🡗🡆🡘🡆🡄🡓🡙🡘
🡁🡑🡆 🡓🡘🡅🡆🡄 🡆🡃🡃🡆🢇 🡓🡇 🡆🡘🡗🡲 🡘🡻🡳 🡙🡘🡲 🡒🡱🡆🡳 🡇🡙 🡄🡑🡆 🡃🡆🡕
🡀🡆🡃🡇🡆 🡙🡘 🡄🡑🡆 🡓🡘🡅🡆🡄 🡆🡃🡃🡆🢇 🡓🡇 🡆🡒🡱🡳 🡙🡘🡲 🡘🡻🡳 🡘🡗🡆🡖

Status	Finished
Started	Sunday, 12 January 2025, 8:26 PM
Completed	Sunday, 12 January 2025, 8:36 PM
Duration	9 mins 43 secs

Question **1**

Correct

Marked out of 1.00

Flag question

Given an array of integers, reverse the given array in place using an index and loop rather than a built-in function.

Example

arr = [1, 3, 2, 4, 5]

Return the array *[5, 4, 2, 3, 1]* which is the reverse of the input array.

Function Description

Complete the function *reverseArray* in the editor below.

reverseArray has the following parameter(s):

int arr[n]: an array of integers

Return

int[n]: the array in reverse order

Constraints

$1 \le n \le 100$

$0 < arr[i] \le 100$

Input Format For Custom Testing

The first line contains an integer, *n*, the number of elements in *arr*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer, *arr[i]*.

```
 */
int* reverseArray(int n, int *a, int *rC) {
    *rC = n;

    int *b = (int*) malloc(sizeof(int)*n);
    for (int i = 0; i < n; i++) {
        b[i] = a[n-i-1];
    }
    return b;
}

```

	Test	Expected	Got	

✓	int arr[] = {1, 3, 2, 4, 5};
	int result_count;
	int* result = reverseArray(5, arr, &result_count);
	for (int i = 0; i < result_count; i++)
	printf("%d\n", *(result + i));
		5
		4
		2
		3
		1
			5
			4
			2
			3
			1
				✓

Passed all tests! ✓

Q2) ↳↔ ⇨▶◀↕↘⇨◀⇨⇦ ⇨▶◀◀↓↔→ ↘⇨⇨↑↓↔⇨ ↓▽ ▶▽⇨⇦ ◀↕ ⇨▶◀ △↕⇦▽ ↓↔◀↕ ▽⇨→↘⇨↔◀▽↘ ^↑⇨ ⇨▶◀◀↓↔→ ↘⇨⇨↑↓↔⇨ ⇨⇨↔ ↕↔↙◢ ↑↕↙⇦ ⇨ △↕⇦ ↕← ↘↓↔∩⇨↔→◀↑ ↕△ ↘↕△⇨↓ ⇨↔⇦ ↓◀ ⇨⇨↔ ↕↔↙◢ ↘⇨↗⇨ ↕↔⇨ ⇨▶◀ ⇨◀ ⇨ ◀↓↘⇨↘ ↓↑↓◁⇨↔ ◀↑⇨ ⇨△↖ △⇨◢ ↙⇨↔→◀↑▽⇨⇨ △⇨▲△⇨▽⇨↔◀↓↔→ ◀↑⇨ ⇦⇨▽↓△⇨⇦ ↙⇨↔→◀↑▽ ↕← ⇨⇨⇨↑ ▽⇨→↘⇨↔◀↓ ⇦⇨◀⇨△↘↓↔⇨ ↓← ↓◀ ↓▽ ▲↕▽▽↓⇆↙⇨ ◀↕ ↘⇨↗⇨ ◀↑⇨ ↔⇨⇨⇨▽▽⇨△◢ ⇨▶◀▽ ▶▽↓↔→ ◀↑↓▽ ↘⇨⇨↑↓↔⇨↘ ^↑⇨ △↕⇦ ↓▽ ↘⇨△↗⇨⇦ ↓↔◀↕ ↙⇨↔→◀↑▽ ⇨↙↖ △⇨⇨⇦◢↓ ↓↔ ◀↑⇨ ↕△⇦⇨△ →↓◁⇨↔↘

↓↑◣⇨↘▲↙⇨

↔ ⮥ ⇠

↙⇨↔→◀↑▽ ⮥ ⇨⇢ ⇠ ↕⇨

↘↓↔∩⇨↔→◀↑ ⮥ ↯

^↑⇨ △↕⇦ ↓▽ ↓↔↓◀↓⇨↙↙◢ ▽▶↘↘↙⇨↔→◀↑▽⇠ ⮥ ⇢ ⇁ ⇠ ⇁ ↕ ⮥ ↳ ▶↔↓◀▽ ↙↕↔→↘ ↹↓△▽◀ ⇨▶◀ ↕←← ◀↑⇨ ▽⇨→↘⇨↔◀ ↕← ↙⇨↔→◀↑ ⇢ ⇁ ⮥ ↯ ↙⇨⇨◁↓↔→ ⇨ △↕⇦ ↳ ↖ ↯ ⮥ ↕↘ ^↑⇨↔ ⇨↑⇨⇨↗ ◀↑⇨◀ ◀↑⇨ ↙⇨↔→◀↑ ↯ △↕⇦ ⇨⇨↔ ⇆⇨ ⇨▶◀ ↓↔◀↕ ▽⇨→↘⇨↔◀▽ ↕← ↙⇨↔→◀↑▽ ⇢ ⇨↔⇦ ⇜↘ ⍒↓↔⇨⇨ ↯ ↓▽ →△⇨⇨◀⇨△ ◀↑⇨↔ ↕△ ⇨▼▶⇨↙ ◀↕ ↘↓↔∩⇨↔→◀↑ ⮥ ⇵ ◀↑⇨ ←↓↖ ↔⇨↙ ⇨▶◀ ⇨⇨↔ ⇆⇨ ↘⇨⇦⇨↘ ↻⇨◀▶△↔ →↻↕▽▽↓⇆↙⇨→↘

↓↑◣⇨↘▲↙⇨

↔ ⮥ ⇠

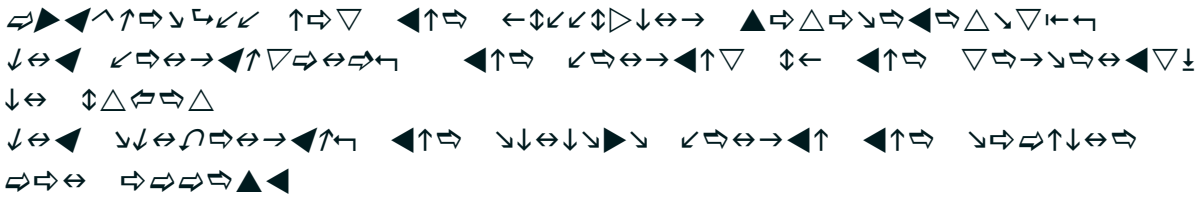
↙⇨↔→◀↑▽ ⮥ ⇨⇢ ↕↓ ⇠⇨

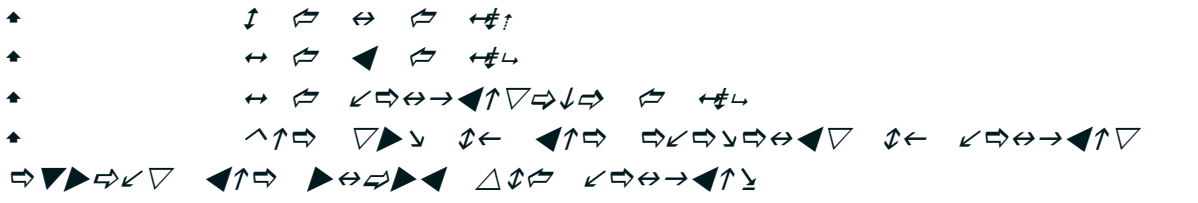
↘↓↔∩⇨↔→◀↑ ⮥ ↯

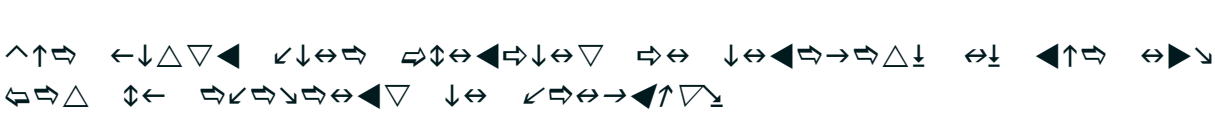
^↑⇨ △↕⇦ ↓▽ ↓↔↓◀↓⇨↙↙◢ ▽▶↘↘↙⇨↔→◀↑▽⇠ ⮥ ⇢ ⇁ ↕ ⇁ ⇠ ⮥ ↳ ▶↔↓◀▽ ↙↕↔→↘ ⇉↔ ◀↑↓▽ ⇨⇨▽⇨↓ ◀↑⇨ ↓↔↓◀↓⇨↙ ⇨▶◀ ⇨⇨↔ ⇆⇨ ↕← ↙⇨↔→◀↑ ⇢ ↕△ ⇢ ⇁ ↕ ⮥ ⇂↘ ↻⇨→⇨△⇦↙⇨▽▽ ↕← ◀↑⇨ ↙⇨↔→◀↑ ↕← ◀↑⇨ ←↓△▽◀ ⇨▶◀↓ ◀↑⇨ △⇨↘⇨↓↔↓↔→ ▲↓⇨⇨⇨ ▷↓↙↙ ⇆⇨ ▽↑↕△◀⇨△ ◀↑⇨↔ ↘↓↔∩⇨↔→◀↑↘ ⇉⇨⇨⇨▶▽⇨ ↔ ↖ ↔ ⮥ ↕ ⇨▶◀▽ ⇨⇨↔↔↕◀ ⇆⇨ ↘⇨⇦⇨↓ ◀↑⇨ ⇨↔▽▷⇨△ ↓▽ →⇉↘◢↕▽▽↓↖ ⇆↙⇨→↘

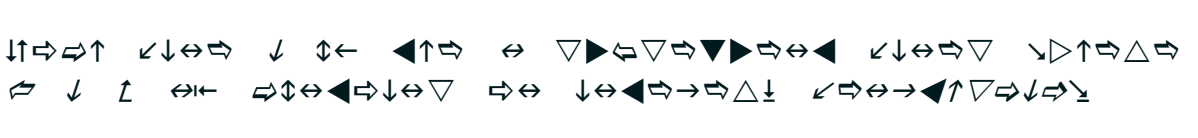
↹▶↔⇨◀↓↕↔ ⇆⇨▽⇨△↓▲◀↓↕↔

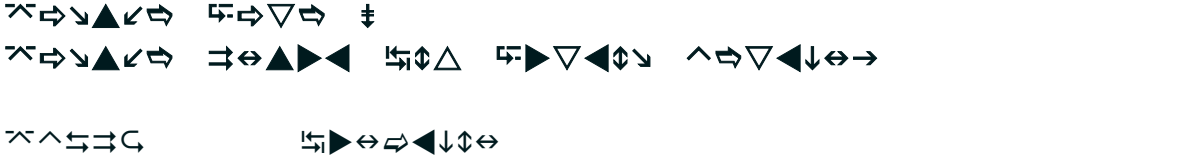
⇋↕↘▲↙⇨◀⇨ ◀↑⇨ ←▶↔⇨◀↓↕↔ ⇨▶◀^↑⇨↘↳↙↙ ↓↔ ◀↑⇨ ⇨⇦↓◀↕△ ⇆⇨↙↕▷↘

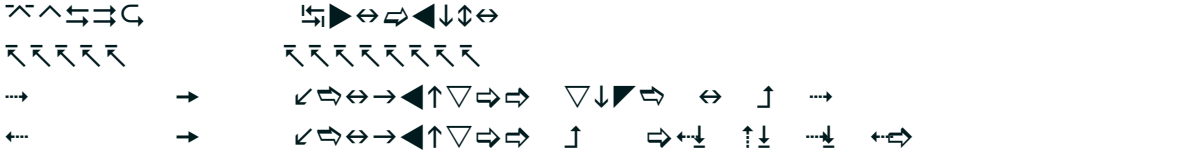












⇠
↳ → ↘↓⇔↰⇨⇔→◀↑↱ ↳

🡵⇨↘▲↙⇨ ↻▶◀▲▶◀

↻↕▽▽↓⇦↙⇨

⇵◣▲↙⇨⇔⇨◀↓↕⇔

🡹↑⇨ ▶⇔⇨▶◀ △↕⇦ ↓▽ ⇠ 🡳 🡱 🡳 ⇢ 🡳 ⇠ ↱ ↬ ▶⇔↓◀▽ ↙↕⇔→↘
🡸▶◀ ◀↑⇨ △↕⇦ ↓⇔◀↕ ↙⇨⇔→◀↑▽ ↕← ⇠ 🡳 🡱 🡳 ⇢ ↱ ↬ ⇨⇔⇦ ⇢↘
🡹↑⇨⇔ ⇨▶◀ ◀↑⇨ ↬ ▶⇔↓◀ ▲↓⇨⇨⇨ ↓⇔◀↕ ↙⇨⇔→◀↑▽ ⇠ ⇨⇔⇦ 🡱
🡳 ⇢ ↱ ↳↘ 🡹↑⇨ △⇨↘⇨↓⇔↓⇔→ ▽⇨→↘⇨⇔◀ ↓▽ 🡱 🡳 ⇢ ↱
↳ ▶⇔↓◀▽ ⇨⇔⇦ ◀↑⇨◀ ↓▽ ↙↕⇔→ ⇨⇔↕▶→↑ ◀↕ ↘⇨↗⇨ ◀↑⇨ ←↓↖
⇔⇨↙ ⇨▶◀↘

🡵⇨↘▲↙⇨ 🡸⇨▽⇨ ↔
🡵⇨↘▲↙⇨ ⇉⇔▲▶◀ 🡷↕△ 🡸▶▽◀↕↘ 🡹⇨▽◀↓⇔→

🡵🡹⇆⇉↻ 🡷▶⇔⇨◀↓↕⇔
🡼🡼🡼🡼🡼 🡼🡼🡼🡼🡼🡼🡼🡼
⇠ → ↙⇨⇔→◀↑▽⇨⇨ ▽↓◤⇨ ⇔ ↱ ⇠
⇡ → ↙⇨⇔→◀↑▽⇨⇨ ↱ ⇨⇡↓ ⇣↓ ↕⇨
⇣
↕
↭ → ↘↓⇔↰⇨⇔→◀↑↱ ↭

🡵⇨↘▲↙⇨ ↻▶◀▲▶◀

⇉↘▲↕▽▽↓⇦↙⇨

⇵◣▲↙⇨⇔⇨◀↓↕⇔

🡹↑⇨ ▶⇔⇨▶◀ △↕⇦ ↓▽ 🡱 🡳 🡵 🡳 ↕ ↱ ↬ ▶⇔↓◀▽ ↙↕⇔→↘ ↳←🡼
◀⇨△ ↘⇨↗↓⇔→ ⇨↓◀↑⇨△ ⇨▶◀↓ ◀↑⇨ △↕⇦ ▷↓↙↙ ⇦⇨ ◀↕↕ ▽↑↕△◀
◀↕ ↘⇨↗⇨ ◀↑⇨ ▽⇨⇨↕⇔⇦ ⇨▶◀↘

Question **2**
Correct
Marked out of 1.00
Flag question

An automated cutting machine is used to cut rods into segments. The cutting machine can only hold a rod of *minLength* or more, and it can only make one cut at a time. Given the array *lengths[]* representing the desired lengths of each segment, determine if it is possible to make the necessary cuts using this machine. The rod is marked into lengths already, in the order given.

Example

n = 3
lengths = [4, 3, 2]
minLength = 7

The rod is initially *sum(lengths) = 4 + 3 + 2 = 9* units long. First cut off the segment of length *4 + 3 = 7* leaving a rod *9 - 7 = 2*. Then check that the length *7* rod can be cut into segments of lengths *4* and *3*. Since *7* is greater than or equal to *minLength = 7*, the final cut can be made. Return "*Possible*".

Example

n = 3
lengths = [4, 2, 3]
minLength = 7

The rod is initially *sum(lengths) = 4 + 2 + 3 = 9* units long. In this case, the initial cut can be of length *4* or *4 + 2 = 6*. Regardless of the length of the first cut, the remaining piece will be shorter than *minLength*. Because *n - 1 = 2* cuts cannot be made, the answer is "*Impossible*".

Function Description

Complete the function *cutThemAll* in the editor below.

cutThemAll has the following parameter(s):
int lengths[n]: the lengths of the segments, in order
int minLength: the minimum length the machine can accept

Returns
string: "*Possible*" if all *n-1* cuts can be made. Otherwise, return the string "*Impossible*".

Constraints

- $2 \le n \le 10^5$
- $1 \le t \le 10^9$
- $1 \le lengths[i] \le 10^9$
- *The sum of the elements of lengths equals the uncut rod length.*

Input Format For Custom Testing

The first line contains an integer, *n*, the number of elements in *lengths*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer, *lengths[i]*.

The next line contains an integer, *minLength*, the minimum length accepted by the machine.

```c
#include <stdlib.h>

int cmp(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}

char* cutThemAll(int n, long *a, long mL) {

    int s = 0;

    for (int i = 0; i < n;i++) {
        s+=a[i];
    }

    long r = s;
    qsort(a, n, sizeof(long), cmp);

    for (int i = 0; i < n; i++) {
        if (r == mL) {
            return "Possible";
        }

        if (r > mL) {
            r-=a[i];
        }else {
            return "Impossible";
        }
    }

    return "Possible";

}

```

	Test	Expected	Got	
✓	`long lengths[] = {3, 5, 4, 3};` `printf("%s", cutThemAll(4, lengths, 9))`	Possible	Possible	✓
✓	`long lengths[] = {5, 6, 2};` `printf("%s", cutThemAll(3, lengths, 12))`	Impossible	Impossible	✓

Passed all tests! ✓